STAGE 1 . CONCEPT DEEP DIVE . SCRUM-2 (#8)

Maven / sbt vs pip -- why py4j's version is hardcoded

The question behind review item #8: Java/Scala projects manage library versions with Maven or sbt. Why does this project hardcode a literal filename instead? Does Python have an equivalent, and why isn't this project using it yet.

01 What Maven / sbt actually do

Both exist to answer one question: get the exact right version of a library without hunting for the file by hand. You write a manifest -- pom.xml for Maven, build.sbt for sbt -- declaring what you want, and the tool fetches it, plus anything that library itself needs ("transitive dependencies"), from a shared repository (Maven Central). You never type a specific jar's filename yourself.

```
<!-- pom.xml -->
<dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
    <version>3.4.0</version>
</dependency>
```

Maven reads this, fetches spark-core-3.4.0.jar and every jar it depends on into a local cache (~/.m2/repository), and assembles the classpath. No filename ever appears in your own code -- Maven owns that detail.

02 Python's parallel: pip + PyPI

Same idea, different names. requirements.txt or pyproject.toml is the manifest (the pom.xml equivalent). PyPI (Python Package Index) is the shared repository (the Maven Central equivalent). pip is the tool that reads the manifest and fetches.

```
# requirements.txt
pyspark==3.4.0
```

pip install -r requirements.txt fetches pyspark, and -- because PySpark's own PyPI listing declares it as a dependency -- the one exact matching version of py4j that release needs. No version-suffixed filename appears anywhere in your code either.

03 Proof, from this exact machine

This project's local .venv already has pyspark and py4j pip-installed -- run .venv/bin/pip freeze and this is what comes back:

```
py4j==0.10.9.7
pyspark==3.4.0
```

> **THE POINT**
>
> Nobody typed "0.10.9.7" anywhere to get this pairing. pip resolved it purely from installing pyspark==3.4.0 -- and it landed on the exact same version that's hand-typed into docker-compose.yml's PYTHONPATH. Same answer, two different ways of getting there -- only one of which updates itself automatically the moment the pyspark version changes.

04 Why this project has neither pyproject.toml nor requirements.txt yet

Two separate reasons, not one:

(a) Docker side: the Spark container never runs pip install for pyspark at all. The image is

apache/spark-py:v3.4.0 -- a pre-built image that already contains a full Spark install (JVM engine, matching Python files, matching py4j) sitting inside it. docker-compose.yml just points PYTHONPATH straight at files already in that image, by exact filename, instead of asking pip to fetch anything. There was never a pip install step here that a manifest would describe.

(b) Local side (.venv): a different story, and a live one. pytest and pyspark were already pip-installed there, and this very session added fpdf2 and pypdf on top -- with nothing, anywhere, writing that down.

THIS IS #8'S PROBLEM AGAIN, ONE LAYER UP

A dependency exists and is being relied on, with no manifest tracking it. Not hypothetical -- checkable right now with .venv/bin/pip freeze. If .venv were deleted today, there is no file anywhere that says what to reinstall to get a working local environment back.

05 What a manifest for this project would look like (illustrative)

Two formats exist. requirements.txt is the older, simpler one -- just a flat list:

```
# requirements.txt
pytest==9.1.1
fpdf2==2.8.7
pypdf==6.15.0
# pyspark / py4j intentionally NOT listed here -- the actual pipeline always runs
# inside the Docker container, using the Spark image's bundled copy. A local
# pyspark install (already in .venv) is only there for IDE autocomplete/type
# checking, not for running anything.
```

pyproject.toml is the newer, standardized format (PEP 621) that most current Python tooling (Poetry, Hatch, uv, PDM) reads and writes -- the closer sibling to pom.xml, since it can hold project metadata, not just a dependency list:

```
# pyproject.toml
[project]
name = "pyspark-lakehouse-aws"
version = "0.1.0"
dependencies = []

[project.optional-dependencies]
dev = ["pytest==9.1.1", "fpdf2==2.8.7", "pypdf==6.15.0"]
```

06 Loose end: sbt does NOT use a pom.xml

pom.xml is specifically Maven's own XML format. sbt uses its own file, build.sbt, written in Scala rather than XML -- different syntax, but it still resolves dependencies against the same kind of shared repository Maven uses (Maven Central, or a private mirror of it). Same underlying concept, two different manifest languages, often the same actual repository behind them.

WHERE THIS LEAVES #8

This is a discussion doc, not a decision. Adding a manifest isn't urgent while the project is this small -- but the .venv gap is real and present today, not hypothetical: fpdf2 and pypdf are installed and relied on right now with zero record of that fact anywhere in the repo. Worth deciding, separately from #8 itself, whether a requirements.txt for local/dev tooling is worth adding before that drifts further.

Further reading -- official documentation

Maven -- POM Reference

The canonic

`https://maven.apache.org/pom.html`

sbt -- Library Dependencies

How build.sb

`https://www.scala-sbt.org/1.x/docs/Library-Dependencies.html`

pip -- Requirements File Format

The requirer

`https://pip.pypa.io/en/stable/reference/requirements-file-format/`

Python Packaging -- pyproject.toml specification (PEP 621)

The modern

`https://packaging.python.org/en/latest/specifications/pyproject-toml/`

PySpark on PyPI

PySpark's o

`https://pypi.org/project/pyspark/`